

Scoping Statico - Dinamico

Definire brevemente
scoping (statico e dinam.)

```
{ int a = 0; int y = 1;
  (*)
  void exec () { int x;
                (**)
                y = fun();
                a = a + 1;
                {
  while (a <= 1) { exec(); }
}
```

Completare il codice
in modo tale che
in caso di scoping
statico le due esecuzioni
di fun restituiscono lo
stesso valore; in
caso di scoping dinamico
valori diversi

→ Dobbiamo definire fun();

→ Dobbiamo utilizzare una variabile non locale
in fun() che abbia sia un ambiente
nel blocco di definizione sia nel
blocco di chiamato

Scopiamo di usare x → dobbiamo dichiarare e
inizializzare nel blocco di
definizione

→ dobbiamo inizializzare nel
blocco di chiamato

(*) Blocco di def: int x = 2; ←
 int fun () { return x; }

(**) Blocco di chiamato: x = a

Scoping statico: fun() ha come ambiente di
riferimento quello globale la cui
x non viene modificata ⇒ restituisce
sempre 2

Scoping dinamico: fun() ha come ambiente di riferimento quello dentro exec(), quindi $x=0$ ed essendo a una variabile globale vale 0 alla prima chiamata e 1 alla seconda.

SCOPING + BINDING

↳ Def. brevemente scoping (statico/dinamico)

↳ Dire quando servono le regole di binding

Servono le regole di scoping quando una procedura ha un ambiente non locale.

Servono anche le regole di binding quando una procedura con ambiente non locale è passata come parametro ad un'altra procedura

↳ Definire le regole di binding (shallow/deep)

⇒ dire quali sono gli ambienti usati?

Statico

① Ambiente del blocco di definizione

ambiente non locale ⇒ servono le regole di scoping

```
int x=3; int y=2;
int fun1(int m) { return x+y+m; }
```

```
int fun2(int h(int b)) {
```

```
  int x=4; int y=4;
```

```
  return h(b)+x;
```

```
}
```

Shallow (+ dinamico)

③ Ambiente del blocco di chiamata effettiva mediante il parametro attuale

```
;
```

Deep (+ dinamico)

② Ambiente del blocco in cui viene creato il frame tra parametro attuale e formale

```
int z = fun2(fun1);
```

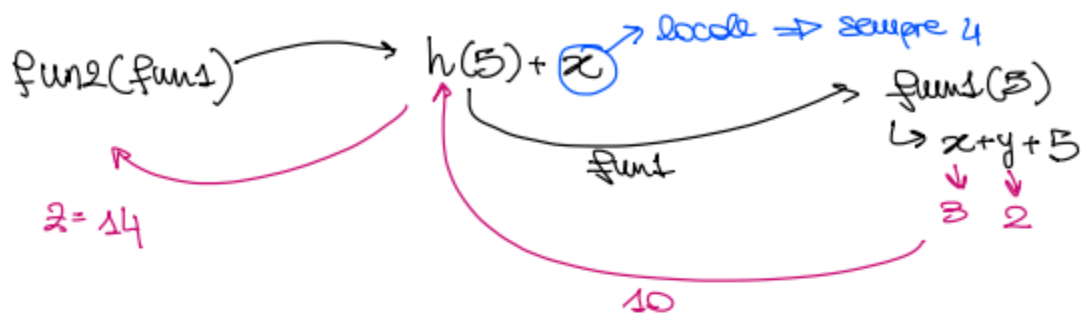
```
}
```

→ servono le regole di binding

fun2 non ha ambiente non locale → non utilizza

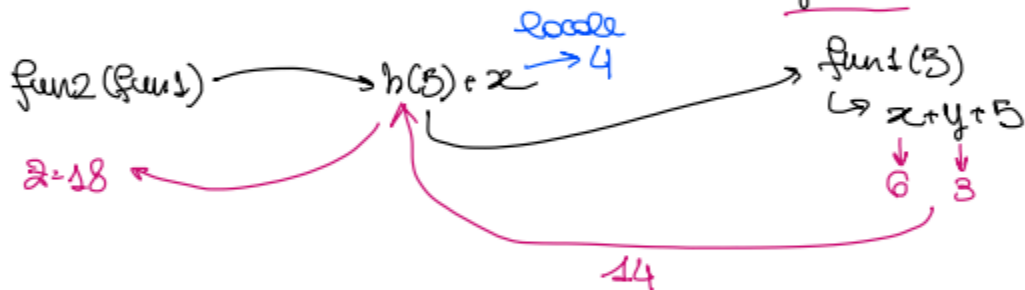
SCOPING STATIC

fun1 → Ambiente di riferimento e' ① $\begin{cases} x=3 \\ y=2 \end{cases}$ (Supponiamo non ci siano modifiche)



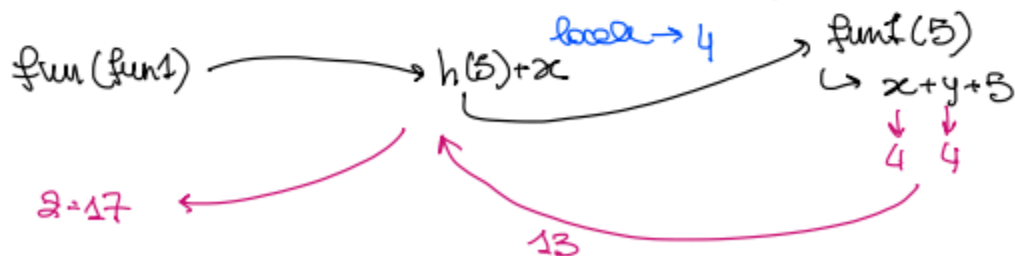
SCOPING DINAMICO + DEEP BINDING

fun1 → Ambiente di riferimento e' ② $\begin{cases} x=6 \\ y=3 \end{cases}$



SCOPING DINAMICO + SHALLOW BINDING

fun1 → Ambiente di riferimento e' ③ $\begin{cases} x=4 \\ y=4 \end{cases}$



SCOPING : IMPLEMENTAZIONE

↳ Def brevemente scoping (statico/dinamico)

↳ Def link statico / catena statica

↳ Def CRT e come e' approssimata

main:

int x; int y;

void fun1() { int z = x + y; }

void fun2() { y = y + 1; }

void fun3() { int y = x + 2;

fun2();

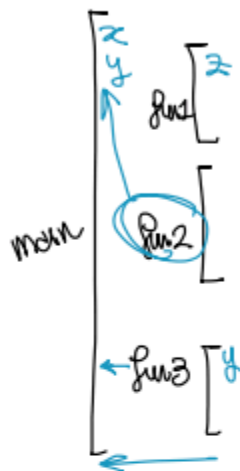
fun1();

;

x = 5; y = 2;

fun3();

;



Sol(main) = 0

Sol(fun1) = Sol(fun2) = Sol(fun3) = 1

Scoping statico

main

x	5
y	2

chiamata main → fun3()

main

x	5
y	2

fun3

y	7
---	---

link statico

Per costruire il link statico dobbiamo calcolare

$$K_{fun3} = Sol(main) - Sol(fun3) + 1 \\ = 0 - 1 + 1 = 0$$

$$CS(fun3) = main$$

Esecuzione $y = x + 2$

↳ Per risolvere x dobbiamo

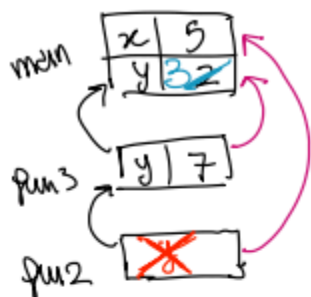
dobbiamo risolvere Calcolare Nx

di 1 la catena statica

$$Nx = Sol(fun3) - Sol(main) = 1$$

procedura più vicina nella struttura di ambiente statica che def x

Chiamata fun3 → fun2



Costruiamo link statico:

$$K_{fun2} = \text{Sol}(\text{fun3}) - \text{Sol}(\text{fun2}) + 1 \\ = 1 - 1 + 1 = 1$$

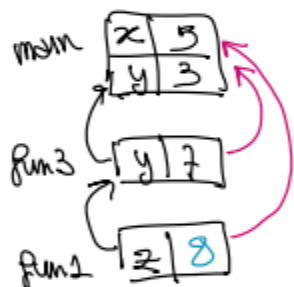
$$CS(\text{fun2}) = CS(\text{fun3}) = \text{main}$$

Esecuzione

$$y = y + 1 \rightarrow 3 \text{ nel main}$$

$$\hookrightarrow N_y = \text{Sol}(\text{fun2}) - \text{Sol}(\text{main}) = 1 \\ \Rightarrow y \text{ e' nel main} = 2$$

Chiamata fun3 → fun1



Costruiamo link statico:

$$K_{fun1} = \text{Sol}(\text{fun3}) - \text{Sol}(\text{fun1}) + 1 \\ = 1$$

$$CS(\text{fun1}) = CS(\text{fun3}) = \text{main}$$

Esecuzione di fun1

$$z = x + y \rightarrow 3 \text{ nel main}$$

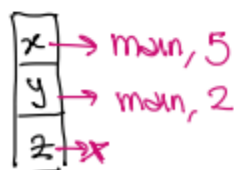
$$N_x = \text{Sol}(\text{fun1}) - \text{Sol}(\text{main}) = 1$$

$$N_y = \text{Sol}(\text{fun1}) - \text{Sol}(\text{main}) = 1$$

Termina tutto

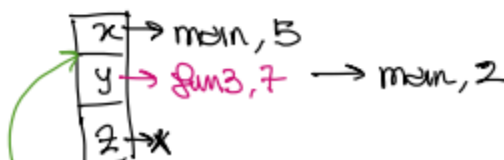
Scoping dinamico

Esempio CRT



Exc main

chiamata main → fun3

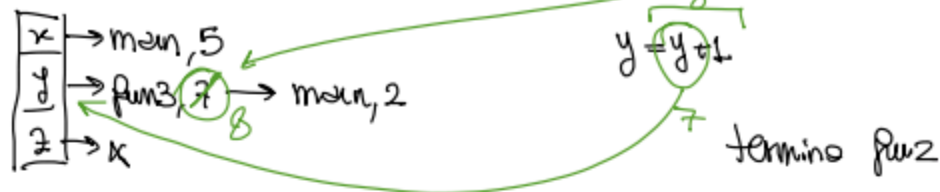


Esecuzione fun3

$$y = x + 2$$

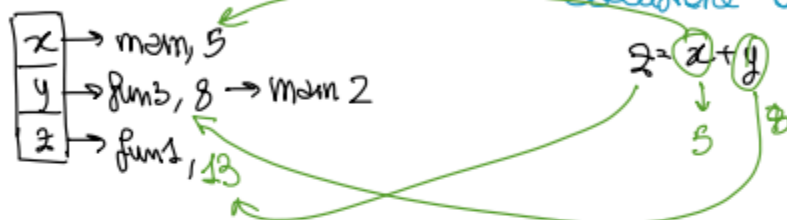
Chiamata fun3 → fun2

Esecuzione di fun2



Chiamata fun3 → fun1

Esecuzione di fun1



Termina tutto

PASSAGGIO DI PARAMETRI

→ def brevemente la tipologie di passaggio di parametri citate nel testo

```
void fun(int a, int b) {
```

```
    a = a + 5;
```

```
    b = 2b;
```

```
}
```

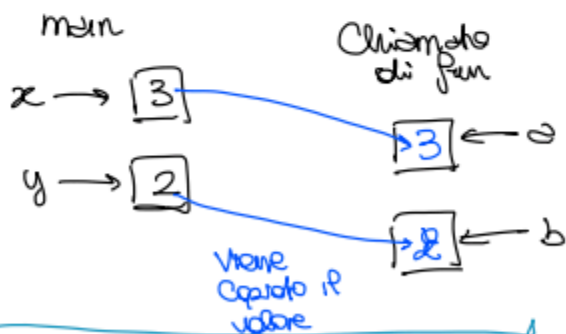
```
void main() {
```

```
    int a=3; int y=2;
```

```
    fun(x, y);
```

```
}
```

Passaggio per valore



Esecuzione di fun

$a \rightarrow 8$

$b \rightarrow 4$

Terminazione di fun

~~$a \rightarrow 8$~~

~~$b \rightarrow 4$~~

Passaggio per riferimento

main

chiamato
di fun

$x \rightarrow [3] \leftarrow a$

$y \rightarrow [2] \leftarrow b$
"copio"
gli indirizzi

$\Rightarrow$ nel main i valori di x
e y restano invariati

Esecuzione di fun

$x \rightarrow [8] \leftarrow a$

$y \rightarrow [2] \leftarrow b$

Terminazione di fun

~~$x \rightarrow [8] \leftarrow a$~~
 ~~$y \rightarrow [2] \leftarrow b$~~

Si eliminano
 a, b
(indirizzi)

$\Rightarrow$ nel main i valori
di x e y sono
stati modificati

RICORSIONE

$\hookrightarrow$ Def. brevemente ricorsione e ricorsione in code

int function (list) {

if length(list) = 0 then return 0;

else return (car(list) + function(cdr(list)));

}

$\uparrow$
non e' in code

function(369) $\leftarrow 18$

$\hookrightarrow 3 + \text{function}(69) \leftarrow 15$

$\hookrightarrow 6 + \text{function}(9) \leftarrow 9$

$\hookrightarrow 9 + \text{function}()$
 $\uparrow$
0

$\Rightarrow$ Il programma calcola ricorsivamente la somma


```
int function (list list) { return functr (list, 0); }
```

```
int functr (list list, int res) {
```

```
    if length (list) = 0 return res;
```

```
    else return functr (cdr (list), car (list) + res);
```

```
}
```

function ((369))



functr ((369), 0)



functr ((6,9), 3+0)



functr ((9), 6+3)



functr ((), 9+9)

18

SEMANTICA

$\rho = [x \mapsto 2, y \mapsto 4]$ → cose è un ambiente dinamico

$\sigma = [4]$ → cose è una memoria

$y := (x+4) * y$

descrivere le derivazioni della
semantica dinamica

→ dire brevemente come
è la sem. dinamica
per assegnamento (comandi)
e per espressioni

Dobbiamo eseguire il comando $y := (x+4) * y$
 $\Rightarrow$ la regola chiede di valutare l'espressione a destra

$$\frac{\rho \vdash \langle (x+4) * y, \sigma \rangle \rightarrow^* \langle \underline{k}, \sigma \rangle \Rightarrow \text{Dobbiamo valutare l'espressione } (x+4) * y}{\rho \vdash \langle \underline{y} := (x+4) * y, \sigma \rangle \rightarrow \langle y := \underline{k}, \sigma \rangle}$$

$\rightarrow$ Valutazione di $(x+4) * y$

$$\frac{\rho \vdash \langle x+4, \sigma \rangle \rightarrow \langle \underline{k_1}, \sigma \rangle}{\rho \vdash \langle \underline{(x+4)} * y, \sigma \rangle \rightarrow \langle \underline{k_1} * y, \sigma \rangle} \Rightarrow \text{Dobbiamo valutare } x+4$$

$\rightarrow$ Valutazione di $x+4$

$$\left. \begin{array}{l} \frac{\rho \vdash \langle x, \sigma \rangle \rightarrow \langle 2, \sigma \rangle}{\rho \vdash \langle \underline{x}+4, \sigma \rangle \rightarrow \langle \underline{2+4}, \sigma \rangle} \quad \rho(x) = 2 \\ \rho \vdash \langle 2+4, \sigma \rangle \rightarrow \langle \underline{k_1}, \sigma \rangle \end{array} \right\} \begin{array}{l} \rho \vdash \langle x+4, \sigma \rangle \rightarrow^* \langle 6, \sigma \rangle \\ \Rightarrow k_1 = 6 \end{array}$$

$\rightarrow$ Valutazione di $6 * y$

$$\frac{\rho \vdash \langle y, \sigma \rangle \rightarrow \langle 4, \sigma \rangle}{\rho \vdash \langle 6 * y, \sigma \rangle \rightarrow \langle \underline{6 * 4}, \sigma \rangle} \quad \rho(y) = e_y \wedge \sigma(e_y) = 4$$

$$\rho \vdash \langle 6 * 4, \sigma \rangle \rightarrow \langle 24, \sigma \rangle$$



$$\rho \vdash \langle (x+4) * y, \sigma \rangle \xrightarrow{*} \langle 2u, \sigma \rangle$$

$$\rho \vdash \langle y := (x+4) * y, \sigma \rangle \rightarrow \langle y := 2u, \sigma \rangle$$

Dobbiamo eseguire $y := 2u$

$$\rho \vdash \langle y := 2u, \sigma \rangle \rightarrow \underbrace{\sigma [p_y \mapsto 2u]}_{\rho(y) = p_y}, \rho(y) = p_y$$

Derivazione semantica statica (supponiamo che venisse chiesto la sem. statica)

$$\Delta = [y \mapsto \text{intloc}, x \mapsto \text{int}] \rightarrow \text{così è ambiente statico}$$

Dobbiamo verificare se $y := (x+4) * y$ è ben formato

$$\frac{\Delta \vdash (x+4) * y : \text{int}}{\Delta \vdash y := (x+4) * y} \quad \Delta(y) = \text{intloc}$$

Supponiamo che Δ che $\Delta(y) = \text{intloc}$ dobbiamo verificare che l'espressione sia int

Dobbiamo verificare $\Delta \vdash (x+4) * y : \text{int}$

$$\frac{\Delta \vdash x+4 : \text{int} \quad \Delta \vdash y : \text{int}}{\Delta \vdash (x+4) * y : \text{int}}$$

vero perché $\Delta(y) = \text{intloc}$
e int se

Dobbiamo verificare $\Delta \vdash x+4 : \text{int}$

$$\frac{\Delta \vdash x : \text{int} \quad \Delta \vdash 4 : \text{int}}{\Delta \vdash x+4 : \text{int}}$$

vero $\Delta(x) = \text{int}$ vero perché assumo

verificato

Fore: $p \mapsto y := 3x + y$ sem. dinamica $\rho = [x \mapsto 3, y \mapsto 4]$
 Semantica $\sigma = [4 \mapsto 2]$

• $p \vdash \text{if } (x > 0) \text{ then } x := x + 1 \text{ else } x := x - 1$

$\rho = [x \mapsto 4]$ $\sigma = [4 \mapsto 2]$

Fore
 ricorsivi : $\text{data fun}(\text{data list}) \{$
 $\text{if length}(\text{list}) = 0 \text{ then return}();$
 $\text{else return concat}(2 * \text{car}(\text{list}), \text{fun}(\text{cdr}(\text{list})));$
 $\}$
 su (369)

Fore
 scoping + binding : $\{ \text{int } a = 4; \text{ int } b = 3; \text{ ①}$
 $\text{int } \text{pippo}(\text{int } m) \{ \text{return } a + b + m; \}$
 $\text{int } \text{pluto}(\text{int } f(\text{int } z)) \{$
 $\text{② int } a = 6;$
 $\text{return } f(b) + a;$
 $\}$
 $\text{int } \text{exec}() \{$
 $\text{② int } b = 2;$
 $\text{return } \text{pluto}(\text{pippo});$
 $\}$
 $\text{int } z = \text{exec}();$
 $\}$

possibile
 come
 parametro
 con ambiente
 non locale
 ↓
 regole di
 binding

① $a = 4 \rightarrow \text{pippo}$
 $b = 3 \rightarrow \text{pluto}$

② $\text{pluto} \rightarrow b = 2$
 $\rightarrow a = 4 \text{ locale}$
 $\text{pippo} \rightarrow b = 2$
 $\rightarrow a = 4$

③ pluto (e' re sup)
 $\text{pippo} \rightarrow a = 6$
 $\rightarrow b = 2$

ha ambiente
 non locale
 ↓
 regole di
 scoping